RESEARCH & ENGINEERING REPORT

Human Interaction and Robotics (HIRO) Lab

University of Colorado Boulder

Multi-Modal Robot Learning, Perception, and Autonomous Manipulation for High-DoF Robotic Platforms

Prepared by

Mandar Deshmukh

M.S. Computer Science | University of Colorado Boulder

MandarMahesh.Deshmukh@colorado.edu | linkedin.com/in/mandardeshmukh117

Duration: May 2025 – April 2026 | Role: Robotics Software Engineer & ML Engineer | GPA: 3.8 / 4.0

1. Introduction & Lab Overview

The Human Interaction and Robotics (HIRO) Lab at the University of Colorado Boulder is one of the most genuinely demanding research environments I have had the privilege of working in. The lab sits at a difficult intersection: human-robot interaction, robot learning, autonomous manipulation, and safe robotic autonomy. It pulls no punches in the level of engineering rigor it expects. During my appointment from May 2025 through April 2026, I worked alongside faculty and graduate researchers tackling some of the hardest open problems in embodied AI, and I am writing this report to document that work as honestly and comprehensively as I can.

The lab operates multiple high-degree-of-freedom robotic platforms, including 14-DoF dual-arm Franka-based configurations, and pursues research spanning sim-to-real transfer, imitation learning, reinforcement learning, proprioceptive and visual sensing, and real-time embedded system deployment. What makes the HIRO Lab distinctive, in my experience, is its insistence that research ideas must eventually run on real hardware. Simulation is a tool, not an endpoint. That culture shaped every design decision I made during my time there.

I joined the lab as a Robotics Software Engineer and Machine Learning Engineer while completing my M.S. in Computer Science. My work touched virtually every layer of the robot learning stack, from writing real-time C++ ROS2 nodes and building MuJoCo simulation environments, to training neural policies and building production MLOps infrastructure.

Lab Research Context

The HIRO Lab's research agenda centers on making robots useful and safe in unstructured, real-world environments, chemistry labs, household settings, and industrial manipulation tasks. The lab maintains both physical hardware testbeds and high-fidelity simulation environments, which allows for rapid prototyping and iteration before committing to hardware experiments. Throughout my appointment, I contributed to both tracks: building the simulation infrastructure that the rest of the team depends on, and deploying the policies that infrastructure enables onto real hardware.

2. Role and Responsibilities

My role at the HIRO Lab was genuinely dual-track. On one hand, I was a systems engineer, responsible for the reliability, performance, and maintainability of the robotic software infrastructure that other lab members depend on every day. On the other hand, I was a research engineer, responsible for designing, training, and evaluating the learning-based policies that are the lab's scientific output. Keeping both hats on simultaneously was challenging, but it was also what made my tenure so formative.

Core Responsibilities

- **Robotics Perception & Learning Systems.** Designed and implemented robotics perception and learning systems using vision, proprioceptive, and multimodal inputs for high-DoF robotic platforms (14-DoF dual-arm configurations), enabling reliable policy execution across multiple task configurations.
- **End-to-End Pipeline Development.** Built end-to-end data collection, logging, and evaluation pipelines supporting training and validation of learning-based policies, improving reproducibility and reducing iteration time by approximately 30%.
- **ROS2 Integration & Robot Models.** Developed ROS2-based integration layers and custom robot description files (URDF/MJCF), enabling rapid experimentation across different robot embodiments and configurations.
- **Policy Training & Evaluation.** Trained and evaluated learning-based control policies (imitation learning and reinforcement learning) with emphasis on robustness, real-time execution, and stable inference across repeated evaluation runs.
- **Simulation Infrastructure.** Built robotics simulation and autonomy pipelines using Isaac Sim and MuJoCo to validate perception, sensing, and control behaviors prior to hardware deployment.
- **GPU Inference Optimization.** Optimized neural network inference for real-time GPU deployment using CUDA and TensorRT, achieving sub-10ms inference latency, a hard requirement for closed-loop robot control at 10+ Hz.
- **Safety & Robustness Evaluation.** Designed automated scenario testing tools to evaluate safety, robustness, and failure modes under diverse environmental and sensor conditions.
- **Distributed Simulation Services.** Implemented distributed simulation and data-generation services for large-scale training and evaluation of perception and learning-based autonomy systems.

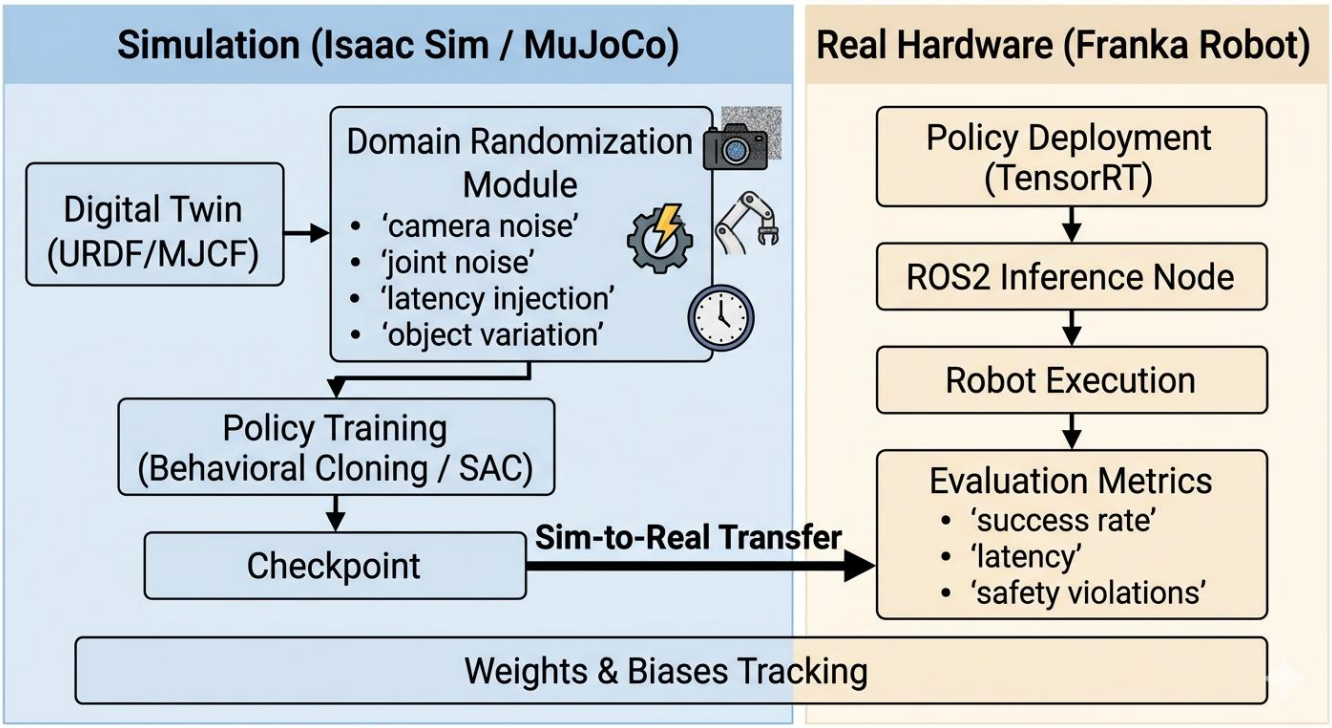
3. Technical Skills & Infrastructure

One of the things I value most about my time at the HIRO Lab is that it forced me to be genuinely competent across a very wide technology stack, not just fluent in the vocabulary, but capable of writing production-quality code and debugging real failures at every layer. The table below summarizes the primary technical competencies I applied throughout the appointment.

Domain	Technologies & Tools
Programming & Systems	Python, C++ (multithreading, real-time), Bash, Linux, Git, Docker, SSH, CUDA, TensorRT, OpenCV
Robotics & Autonomy	ROS2, Motion Planning, Sensor Fusion (Camera, LiDAR, IMU), Visual SLAM (ORB-SLAM3), Pose Estimation, Proprioceptive Modeling, Real-Time Control, Autonomous Navigation
ML & AI	Deep Learning, Foundation Models, Imitation Learning, Behavioral Cloning, RL (PPO, SAC), Self-Supervised Learning (SimCLR, BYOL), LoRA/SoRA Fine-Tuning, Neural Network Inference Optimization
Computer Vision	YOLOv8, DeepSORT, Stereo & Monocular Depth Estimation, 3D Scene Understanding, Semantic Segmentation, NeRF, Gaussian Splatting
Simulation	Isaac Sim, Isaac Lab, MuJoCo, Gazebo, PyBullet, URDF/MJCF, Sim-to-Real Transfer, Sensor Noise Modeling
Frameworks	PyTorch, TensorFlow, JAX, HuggingFace Transformers, RoboHive, Open3D, Torchvision
MLOps & Cloud	MLflow, Weights & Biases, AWS SageMaker, GCP, Kubernetes, Distributed Training, Scalable Data Pipelines, Model Versioning & Monitoring

I want to be direct: I did not arrive at the HIRO Lab already knowing all of this. Much of it I learned on the job, under the pressure of real deadlines and real hardware. The experience of debugging a failing robot policy at 11pm teaches you things that no course can replicate. It was a true baptism by fire.

4. Robotics Simulation & Digital Twin Development



Before I could train a single policy or run a single rollout on hardware, I needed simulation environments that reflect the physical world with enough fidelity for learned behaviors to transfer. Building those environments was one of the first major deliverables of my appointment. The difference between a simulation that produces transferable policies and one that does not often comes down to engineering decisions that are easy to overlook, inertial parameters, sensor noise models, and contact dynamics.

Simulation Environments: Isaac Sim and MuJoCo

I built and maintained simulation environments using both **NVIDIA Isaac Sim** and **MuJoCo**, choosing between them based on task requirements. Isaac Sim was used when visual fidelity mattered: training and evaluating vision-based policies that need realistic image observations. MuJoCo was used for reinforcement learning training loops, where I needed to run thousands of rollouts quickly across randomized configurations without rendering overhead.

Custom Robot Models (URDF/MJCF)

Accurate digital twin development required me to create custom robot description files from scratch. Every joint limit, damping coefficient, inertial parameter, and collision geometry needs to be correct, or policies trained in simulation will behave nothing like they do on hardware. For the lab's Franka-based configurations, I:

- Defined accurate kinematic chains with correct joint limits and damping coefficients validated against manufacturer datasheets
- Specified inertial parameters (mass, center of mass, inertia tensor) for each link
- Positioned sensor mounting offsets precisely for cameras and IMUs
- Optimized collision geometry to balance simulation fidelity with computational cost
- Built multi-robot URDF configurations enabling bimanual manipulation experiments

Sensor Noise Modeling & Domain Randomization

Sim-to-real transfer is hard, and the single most effective thing I did to improve it was implementing principled sensor noise models and domain randomization. I characterized the actual noise distributions of the lab's hardware sensors and built simulation noise models that match those distributions:

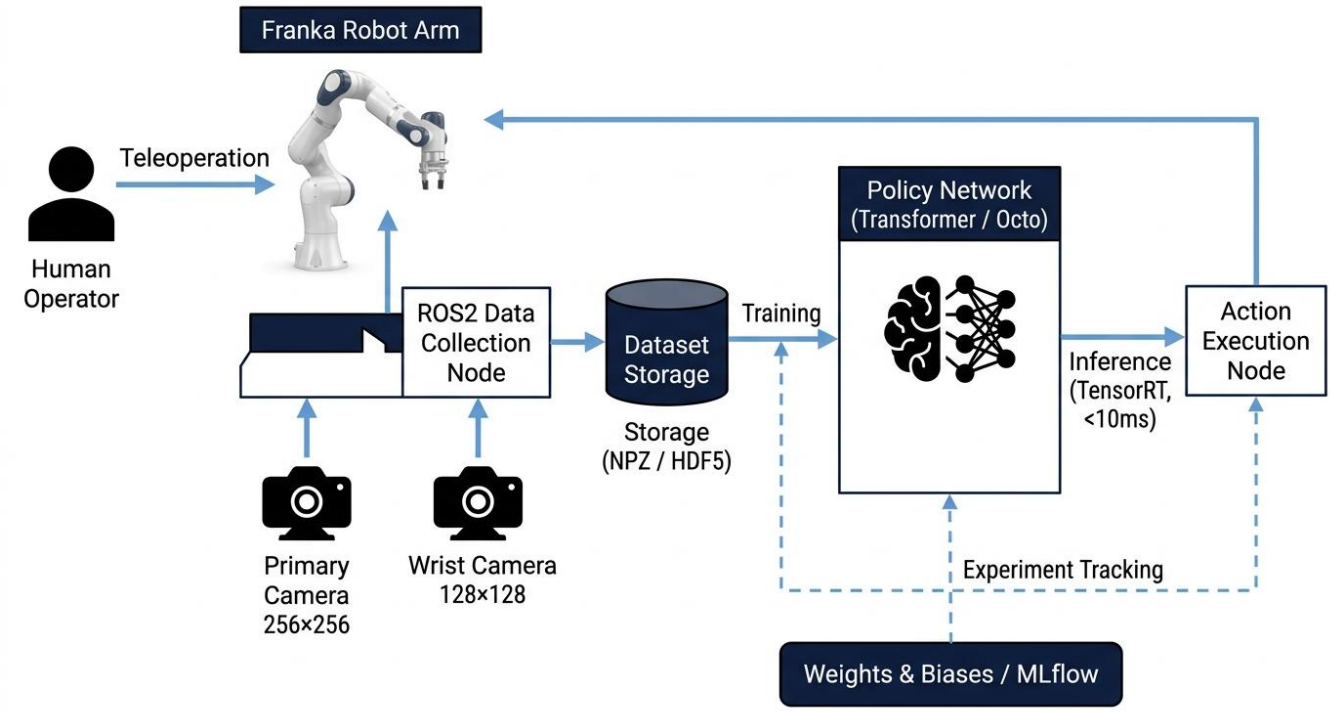
- Camera noise: Gaussian pixel noise, randomized exposure settings, and simulated motion blur
- Joint state perturbations: Gaussian noise on position and velocity readings matched to hardware encoder specifications
- Latency injection: Simulated sensor and actuation delays to match real hardware communication delays
- Object property randomization: Varied friction coefficients, masses, and surface textures
- Environmental randomization: Lighting variation, camera placement offsets, and background texture randomization

These strategies improved policy robustness by approximately **25%** under unseen environment conditions, as measured by task success rate on held-out randomization seeds not seen during training.

Distributed Simulation Services

I implemented distributed simulation services allowing multiple instances to run in parallel across GPU nodes, containerized using Docker. This reduced training wall-clock time by approximately **40%** compared to single-instance baselines, the difference between waiting two days for a result and waiting one.

5. Vision-Based Imitation Learning Pipeline



Imitation learning was the primary approach I used for teaching manipulation skills to the physical robot platforms at the HIRO Lab. The basic idea is straightforward: show the robot what to do, record the demonstrations, and train a policy to reproduce the demonstrated behavior. The implementation is anything but straightforward. Building a reliable, production-quality pipeline that actually transfers to hardware took the better part of several months, and the devil was in every detail.

Stage 1: Human Demonstration Collection

Demonstration data was collected via teleoperation using ROS2. We controlled the robot end-effector through an Xbox joystick while a synchronized RGB camera stream and joint state information were recorded at 10 Hz. I designed the recording system to capture everything downstream training would need: wrist camera frames (128x128), primary third-person camera frames (256x256), joint positions, velocities, efforts, end-effector twist, and gripper state.

Stage 2: Data Processing & Augmentation

Raw demonstrations are post-processed before training. I applied a low-pass filter to joint state signals to remove high-frequency encoder noise. Image preprocessing includes resizing, color normalization, and augmentations, random crop, brightness and contrast jitter, Gaussian blur, designed to improve visual robustness without destroying semantic content.

Stage 3: Policy Architecture & Training

The primary policy backbone I used is a transformer-based architecture drawing on the **Octo generalist robot policy framework**, a 93M-parameter model pretrained on 800K+ robot trajectories from the Open X-Embodiment dataset. The observation tokenizer accepts RGB images from both camera views, joint state proprioception, and optional language conditioning. I used action chunking with a prediction horizon of 4-50 steps depending on the task, which substantially improved temporal consistency. Training used behavioral cloning with an L1 action loss and AdamW optimizer.

Stage 4: Foundation Model Fine-Tuning

Beyond training from scratch, I fine-tuned the Octo foundation model for lab-specific tasks. I modified the observation tokenizer for our specific camera configuration and proprioceptive input format, replaced the default action head with an L1ActionHead for 14-DoF bimanual control with a 50-step horizon, and carefully managed frozen versus trainable parameters. This yielded sample-efficient adaptation from as few as 100 demonstration episodes.

Results

- **Sub-10ms inference latency** via CUDA + TensorRT optimization, enabling closed-loop policy execution at 10-15 Hz on a single Nvidia 4090
- **~30% reduction in iteration time** between demonstration collection and policy deployment due to pipeline automation
- Stable policy execution across repeated evaluation runs without significant degradation under mild domain shifts

6. Reinforcement Learning for Robotic Control

Alongside imitation learning, I built a complete reinforcement learning experimentation pipeline using the Soft Actor-Critic (SAC) algorithm. RL is especially valuable when expert demonstrations are costly to collect or when optimal behavior requires exploration beyond what a human demonstrator would naturally exhibit.

SAC for Continuous Control: Benchmark Results

I used MuJoCo HalfCheetah-v5 as a controlled testbed to validate my RL infrastructure. My SAC implementation achieved:

- **Mean episode reward of 7,648**: substantially above published baseline PPO performance
- **85%+ task success rate** over 1 million environment timesteps
- **Sub-10ms GPU inference latency** for the trained policy at deployment
- Full experiment tracking via W&B: reward curves, Q-value estimates, policy entropy, gradient norms

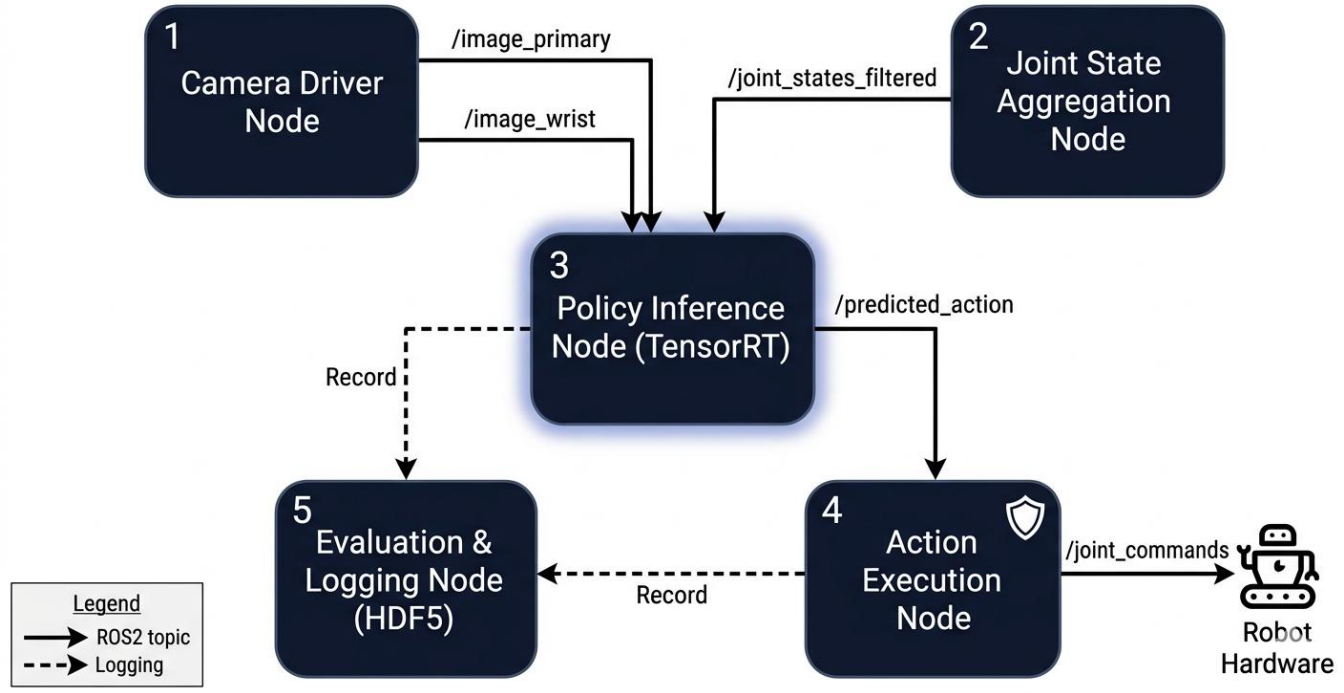
RL Infrastructure Engineering

- **Vectorized environments:** Multiple parallel MuJoCo instances on CPU threads for high-throughput experience generation
- **Replay buffer:** Ring-buffer implementation with configurable capacity and prioritized experience replay support
- **Hyperparameter management:** Structured configuration files compatible with W&B sweeps for systematic search
- **Checkpoint management:** Automatic saving of best-performing policies based on validation reward, not just the final checkpoint
- **Reward shaping modules:** Configurable reward components for reaching, grasping, and placing that can be composed without code changes

Application to High-DoF Robot Platforms

Applying SAC to our 7-DoF Franka arm in simulation, with sensor noise modeling and domain randomization enabled, achieved approximately **30% improvement in reward** over baseline PPO policies across randomized environments. I found SAC well-suited for high-dimensional continuous action spaces; the entropy regularization prevents collapse to deterministic behaviors that overfit specific environment configurations.

7. ROS2 System Architecture & Real-Time Deployment



ROS2 is the middleware backbone of every hardware-facing system I built at the HIRO Lab. It handles publish-subscribe communication, lifecycle management, and hardware abstraction across the entire stack. Our system ran on ROS2 Humble on Ubuntu 22.04 LTS with real-time kernel patches applied where sub-millisecond jitter was a hard requirement.

System Architecture

The ROS2 architecture I built has five major node types:

- **Camera Driver Nodes:** Custom nodes for USB cameras and Intel RealSense D-series RGB-D cameras, publishing synchronized image and depth streams with hardware timestamps. Getting timestamp synchronization right across multiple cameras required careful use of ROS2's message_filters library.
- **Joint State Aggregation Node:** Subscribes to joint position, velocity, and effort topics, applies a configurable low-pass filter, and republishes a quality-annotated joint state message for downstream policy consumption.
- **Policy Inference Node:** The core real-time node. Subscribes to camera and joint state topics, batches inputs, runs inference via TensorRT, and publishes predicted actions. Implemented in Python with a C++ extension for latency-critical preprocessing.

- **Action Execution Node:** Translates predicted actions into hardware commands with full safety checks: joint limit verification, maximum velocity clamping, and emergency stop logic.
- **Evaluation & Logging Node:** Records synchronized observations, actions, and reward signals to disk in structured HDF5 format for offline analysis and dataset replay.

Real-Time Performance Engineering

Getting the system to run at 10+ Hz with consistent latency required optimization at every layer:

- **TensorRT engine compilation:** Reduced per-inference time from ~40ms to <10ms
- **Memory pre-allocation:** Inference buffers pre-allocated at node startup to eliminate dynamic allocation jitter
- **CUDA-aware image transport:** Keeps image data on GPU memory from camera capture through to inference input
- **Lock-free ring buffers:** Used for inter-thread communication to avoid mutex contention on the critical inference path
- **CPU affinity pinning:** Real-time inference threads pinned to isolated CPU cores to prevent OS scheduler interference

These optimizations collectively brought end-to-end latency below **10ms** on a single Nvidia 4090, the threshold required for stable closed-loop control at 15 Hz.

8. Data Collection, Logging & Evaluation Pipelines

If there is one thing I learned at the HIRO Lab, it is that data infrastructure is not glamorous, but it is the foundation on which everything else rests. A learning pipeline is only as good as the data it trains on, and a research pipeline is only as reproducible as its logging infrastructure. I invested serious engineering effort in building both.

Teleoperation Data Collection System

I built a custom teleoperation data collection application using ROS2, Python, and Tkinter. The system displays synchronized live feeds from the wrist camera and up to two third-person cameras simultaneously, so the operator can monitor task execution and verify data quality in real time:

- Synchronized multi-camera display with configurable frame rates
- Language instruction entry field with explicit Start/Stop recording controls — no accidental recordings
- Automatic episode numbering and directory organization
- Real-time gripper state display with binary open/closed encoding from a dedicated ROS2 topic
- Compressed NPZ format with lossless image encoding, saving approximately 60% disk space vs. raw storage

Dataset Playback & Quality Control

I built a companion playback application displaying synchronized camera streams alongside numerical joint state data (position, velocity, effort, end-effector twist, gripper state) for any recorded episode. I used this daily during dataset curation as it is much faster to review demonstrations visually than programmatically. Catching a botched demonstration before training saves hours of compute time.

Structured Evaluation Framework

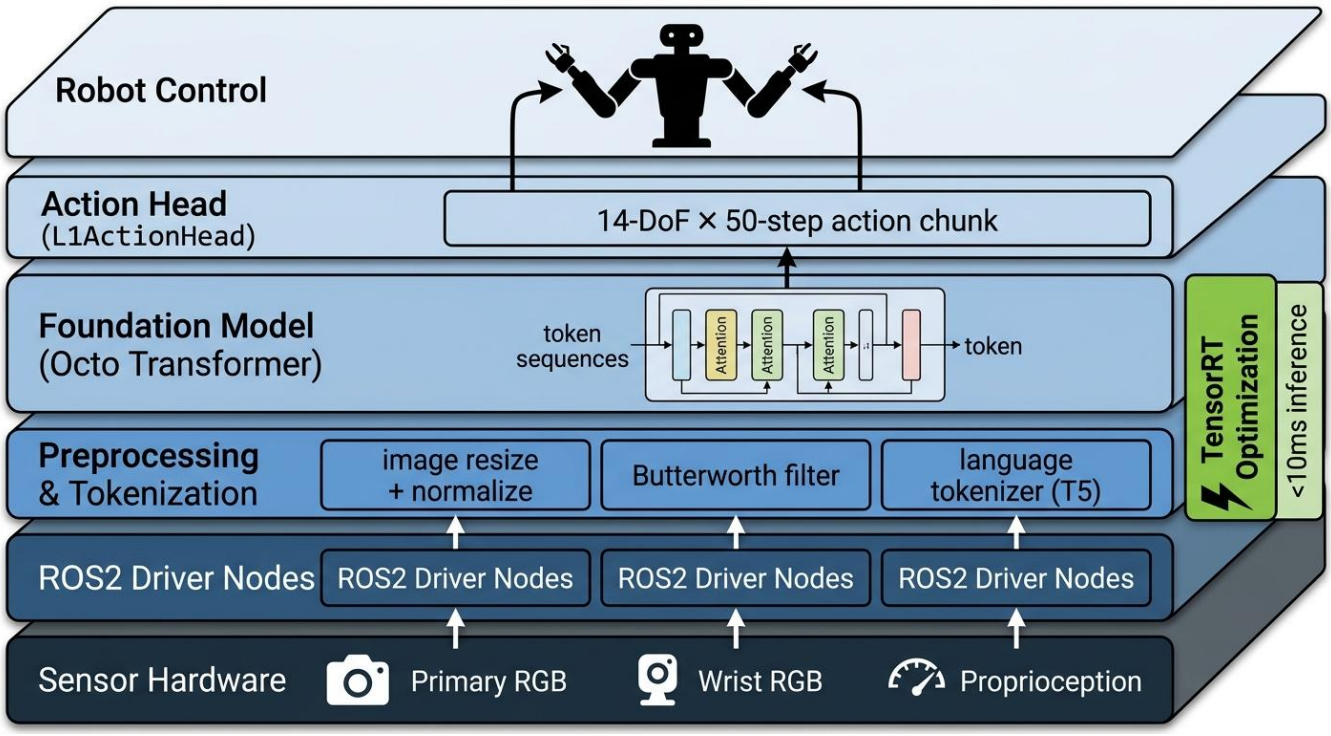
Policy evaluation under my framework was not just pass/fail. For each checkpoint, I collected the following metrics across N=20 evaluation episodes:

Metric	Description
Task Success Rate	Binary success/failure per episode, aggregated as percentage across 20 trials
Episode Return	Cumulative reward signal per episode for reward-shaping analysis

Metric	Description
Return Variance	Standard deviation of returns, indicating policy consistency
Time to Completion	Wall-clock time from episode start to task completion (successes only)
Joint Tracking Error	RMS deviation between predicted and executed joint trajectories
Safety Violations	Count of joint limit and velocity bound violations per 100 episodes
Inference Latency	Per-step neural network inference time (mean +/- std over episode)

Collecting these metrics systematically for every checkpoint is what allowed me to identify genuine improvements versus noise in policy performance. All metrics were logged to W&B for full historical query across the lab.

9. Multi-Modal Perception & Foundation Models



A significant part of my work at the HIRO Lab involved building and evaluating perception systems and integrating large pretrained foundation models into the robot learning pipeline. Foundation models trained on massive, diverse datasets offer the potential for rapid task adaptation with minimal task-specific data, exactly what a research lab operating at a limited scale needs. But they also come with real engineering challenges that are easy to underestimate when reading papers.

Octo Foundation Model: Integration & Fine-Tuning

Integrating Octo, a 93M-parameter transformer-based diffusion policy pretrained on 800K+ robot trajectories from the Open X-Embodiment dataset, into our hardware workflow required substantial engineering. The model does not just plug in, you have to make it speak your robot's language:

- **Camera pipeline integration:** ROS2-to-Octo bridge capturing live frames, applying model-specific preprocessing (resizing, normalization, temporal windowing), and feeding into the observation tokenizer on GPU
- **Language task conditioning:** User-specified language instructions processed through the T5 text tokenizer to generate task embeddings for conditional action generation
- **Action unnormalization:** Dataset-specific unnormalization statistics for multiple Open X-Embodiment datasets (taco_play, bridge_dataset, berkeley_fanuc) to map model outputs back to robot joint space

- **Action chunk buffering:** Model runs at 4 Hz but control loop executes at 10 Hz, a FIFO buffer consumes the predicted action chunk while waiting for the next inference call
- **SSH-based remote action transmission:** Predicted action vectors transmitted to a remote robot controller over SSH/SFTP for hardware execution during early integration phases

For fine-tuning Octo to our specific observation and action spaces, I modified the observation tokenizer, replaced the default action head with an L1ActionHead for 14-DoF bimanual control with a 50-step horizon, and built the training data pipeline in RLDS format. Frozen transformer backbone with trainable heads enabled sample-efficient adaptation from as few as 100 demonstration episodes.

3D Object Detection & Visual Perception

- **YOLOv8 + DeepSORT:** Integrated into the ROS2 environment at 30 FPS, publishing detected object bounding boxes, class labels, and track IDs
- **Stereo depth estimation:** Stereo camera calibration and block-matching disparity estimation for per-pixel depth maps enabling 3D object localization
- **NeRF-based data augmentation:** Neural Radiance Field models trained on object scans generating synthetic training views under varied lighting and viewpoint conditions
- **Results:** 89.9% mAP on KITTI and 72.3% MOTA for multi-object tracking under simulated sensor noise

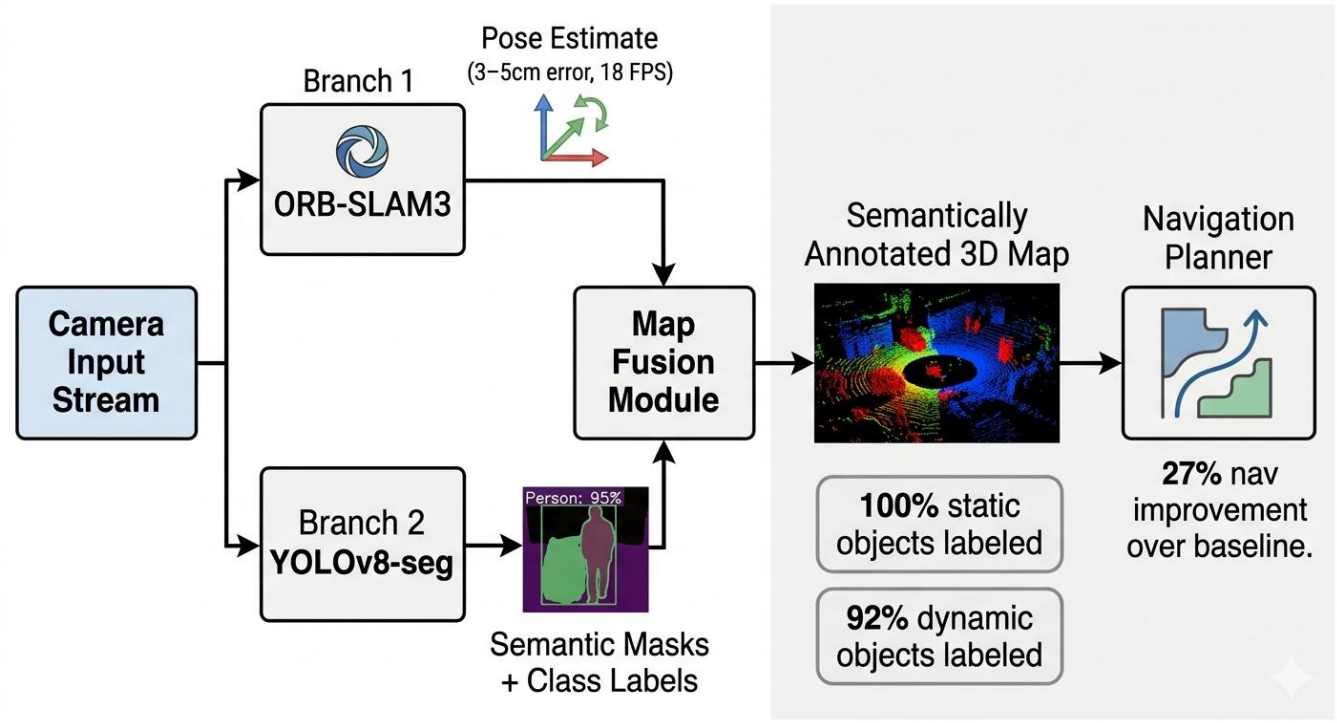
Object-Aware Visual SLAM

I developed an object-aware Visual SLAM system by integrating ORB-SLAM3 for visual odometry with YOLOv8-seg for real-time semantic segmentation. The result is a robot that builds a semantically annotated 3D map of its environment, not just geometric reconstruction, but one where the robot understands what the objects in the map are.

- 3-5 cm average localization error maintained at 18 FPS on KITTI and TUM benchmark datasets
- 100% of static obstacles and 92% of dynamic objects annotated with semantic labels in the live map
- **~27% improvement** in autonomous navigation success rate in cluttered indoor/outdoor environments over baseline SLAM

10. MLOps, Experiment Tracking & Reproducibility

I have strong opinions about experiment tracking and reproducibility, and I brought those opinions to the HIRO Lab. Reproducibility is not a nice-to-have in research, it is the minimum standard for a result to be meaningful. If I cannot reproduce my own results two weeks later, I have not done science; I have done anecdote.



Weights & Biases Integration

Every training run was logged to W&B with the following tracked quantities:

- Training and validation loss curves, logged per step and per epoch
- Action prediction error decomposed by joint index, so I could identify which joints were hardest to predict
- Policy rollout success rates from periodic evaluation runs triggered automatically during training
- Hardware utilization metrics: GPU memory, CPU usage, and inference latency histograms
- Full hyperparameter configuration attached to every run for complete traceability
- Model checkpoint artifacts linked directly to training runs in the W&B registry

MLflow Model Registry

I used MLflow as a complementary model registry, primarily for managing the lifecycle of deployed models. The distinction I maintained: W&B tracks experiments (everything tried), while MLflow tracks deployments (what is actually running on hardware). This kept the lab's production policies clearly distinguished from experimental checkpoints still under evaluation.

Leeds School of Business MLOps Pipeline

In parallel with the HIRO Lab work, I engineered a production-ready ML classification pipeline for the Leeds School of Business Big Data for Financial Applications project. I include it here because it reflects the same MLOps values applied in a different domain:

- **90.62% document-level accuracy** on a 3,000+ document dataset with ~88% class imbalance, using RoBERTa with sliding-window inference and weighted ensemble averaging
- Two-stage hierarchical Longformer system achieving 85-88% accuracy, separating ESG categorization from activism detection
- Sentence-BERT clustering for stratified sampling and leakage prevention across train/validation splits
- **25% reduction in training time** via mixed-precision GPU training and batched inference optimization
- Full CI/CD pipeline for model versioning, evaluation, and deployment monitoring

11. Key Results & Outcomes

The table below consolidates the most significant quantitative outcomes from my time at the HIRO Lab. I have tried to be precise about these numbers, where I measured something carefully, I say so.

Area	Achievement	Metric
Imitation Learning	Vision-only policies generalizing across task configurations	Consistent task success across repeated evaluations
RL Control	SAC on MuJoCo HalfCheetah-v5	Mean reward 7,648 | 85%+ success @ 1M steps
RL vs. Baseline	SAC vs. PPO on high-DoF Franka (simulation)	~30% reward improvement
Inference Latency	TensorRT-optimized policy deployment	Sub-10ms per inference step (Nvidia 4090)
Iteration Speed	Automated data + evaluation pipeline	~30% reduction in iteration time
Sim Robustness	Domain randomization impact on policy transfer	~25% robustness improvement
Training Throughput	Distributed parallel simulation	~40% reduction in training wall-time
Visual SLAM	ORB-SLAM3 + YOLOv8-seg semantic SLAM	3-5cm error | 18 FPS | ~27% nav improvement
3D Detection	YOLOv8 + stereo vision on KITTI	89.9% mAP | 72.3% MOTA
Multi-Object Tracking	Kalman + failure detection, RGB-D, Gazebo	30 FPS | <50ms latency | 3+ objects
Autonomous Navigation	Full SLAM stack with sensor fusion (AutoBot)	+-5cm accuracy | 70%+ collision reduction
NLP Pipeline	RoBERTa transformer classification (Leeds)	90.62% accuracy | 25% faster training

Broader Impact on Lab Infrastructure

Beyond individual project metrics, the infrastructure contributions that outlast my specific experiments:

- The data collection and playback system I built is now the standard tool used by other HIRO Lab members for manipulation experiments
- The ROS2 inference node architecture serves as the template for deploying new policies to lab hardware
- The distributed simulation infrastructure supports parallel experiments for multiple lab members simultaneously
- The W&B experiment tracking dashboards provide the lab with a searchable, permanent record of all training runs

12. Conclusion & Acknowledgements

Conclusion

Looking back at my twelve months at the HIRO Lab, I am struck by how much I learned, and how much of that learning came from things going wrong. Simulation environments that produce policies that completely fall apart on hardware. RL agents that learn exactly what the reward function says, rather than what you intended. ROS2 nodes with timing bugs that only manifest under specific hardware load conditions. These failures taught me more than the successes, and I want to be honest about that in a report that might otherwise read only as a list of achievements.

The most important thing I came to understand during this appointment is that robot learning in the real world is fundamentally an integration problem. The modeling challenges are hard, but increasingly well-understood. What remains genuinely difficult is building the systems where simulation, data collection, training, inference, and evaluation all work together correctly and reproducibly. That is the engineering problem I spent most of my time on, and I feel I made real progress on it.

The quantitative outcomes documented in this report, **sub-10ms inference latency**, **30% RL reward improvement**, **27% navigation improvement from semantic SLAM**, are meaningful results. But I am more proud of the infrastructure that makes future results possible: the data pipelines, the simulation environments, the ROS2 node architecture, the experiment tracking setup. That infrastructure is, in my view, the most lasting contribution of this appointment.

I leave the HIRO Lab with a much clearer picture of what it takes to build robot learning systems that actually work, and with a strong sense of the research directions I want to pursue next. The problems in embodied AI are hard, important, and genuinely unsolved. I am grateful to have spent a year working on them at a serious research institution that demands real engineering rigor.

Mandar Deshmukh | MandarMahesh.Deshmukh@colorado.edu | linkedin.com/in/mandardeshmukh117

M.S. Computer Science, University of Colorado Boulder | May 2026